

Excepciones en Java

Guía de referencia rápida para el manejo de errores en tiempo de ejecución con el mecanismo try-catch de Java.

¿Qué es una excepción? Jerarquía de clases

try · catch · finally Estructura completa

throw vs throws Diferencias clave

Excepciones checked vs unchecked

Excepciones comunes ArithmeticException, etc.

IllegalArgumentException Validaciones

InputMismatchException Entrada por teclado

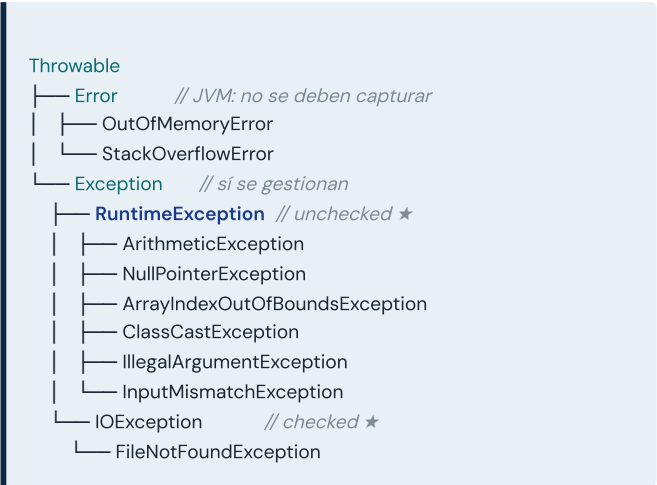
ArrayIndexOutOfBoundsException Acceso a arrays

Buenas prácticas Cuando usar cada una

1 ¿Qué es una excepción?

Un **evento anormal** que interrumpe el flujo normal del programa durante la ejecución. Java usa objetos para representarlas y permite capturarlas para gestionar el error sin que el programa se detenga abruptamente.

JERARQUÍA DE CLASES



CHECKED VS UNCHECKED

Tipo	¿Obliga try-catch?	Ejemplo
CHECKED	Sí, el compilador lo exige	IOException
UNCHECKED	No, pero es recomendable	RuntimeException
ERROR	Nunca capturar	OutOfMemoryError

FLUJO DE EJECUCIÓN



⚠ IMPORTANTE

Si no se captura una excepción, Java imprime el *stack trace* y el hilo de ejecución termina. El bloque finally siempre se ejecuta, incluso si hay un return.

2 try · catch · finally

ESTRUCTURA COMPLETA

```
try {
    // código que puede lanzar excepción
    int resultado = 100 / numero;
    System.out.println("OK: " + resultado);

} catch (ArithmeticException e) {
    // solo si hay excepción de este tipo
    System.out.println("Error: " + e.getMessage());

} catch (Exception e) {
    // captura cualquier otra excepción
    System.out.println("Error genérico");

} finally {
    // SIEMPRE se ejecuta: cerrar recursos
    sc.close();
}
```

COMPORTAMIENTO DE CADA BLOQUE

Bloque	Cuándo se ejecuta
try	Siempre, primero
catch	Solo si hay excepción del tipo indicado
finally	Siempre, haya o no excepción

MULTI-CATCH (JAVA 7+)

```
catch (IOException | SQLException e) {
    // captura ambos tipos a la vez
}
```

EJEMPLO COMPLETO — DIVISIÓN

```
public void dividir(double num, double div) {
    try {
        if (div == 0)
            throw new ArithmeticException("Divisor = 0");
        System.out.println(num / div);
    } catch (ArithmeticException e) {
        System.out.println("Error: " + e.getMessage());
    } finally {
        System.out.println("Fin del cálculo");
    }
}
```

3 throw vs throws

Palabra	Dónde va	Qué hace
throw	Dentro del método	Lanza la excepción en ese instante
throws	Firma del método	Avisa de que el método puede lanzarla

```
// throws → en la firma: AVISA
public void verificar(String s) throws Exception {

    // throw → dentro: LANZA
    if (s == null)
        throw new Exception("Valor nulo");
    }

    // quien llama DEBE usar try-catch
    try {
        verificar(null);
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}
```

ILLEGALARGUMENTEXCEPTION — VALIDACIONES

Se usa cuando un argumento recibido en un método tiene un valor incorrecto.

```
public static void validar(int n) {
    if (n < 0)
        throw new IllegalArgumentException(
            "Negativo: " + n);
    System.out.println("Válido: " + n);
}

// double → no lanza excepción automática
// int → sí lanza ArithmeticException
```

⚠ DOUBLE Y DIVISIÓN POR CERO

5.0 / 0.0 devuelve Infinity — no lanza excepción. Con int: 5 / 0 sí lanza ArithmeticException. Valida manualmente con if (divisor == 0).

4 Excepciones comunes

ARITHMETICEXCEPTION

```
int r = 10 / 0; // ← lanza excepción (solo int)
throw new ArithmeticException("Divisor = 0");
```

ARRAYINDEXOUTOFBOUNDSEXCEPTION

```
int[] arr = {10, 20, 30}; // índices: 0-2
arr[5]; // ← lanza excepción automáticamente
arr[-1]; // ← también fuera de rango
// ✓ Validar: if (i >= 0 && i < arr.length)
```

INPUTMISMATCHEXCEPTION

```
Scanner sc = new Scanner(System.in);
try {
    int n = sc.nextInt(); // lanza si escribes "abc"
} catch (InputMismatchException e) {
    System.out.println("Solo números enteros");
}
```

NULLPOINTEREXCEPTION

```
String s = null;
s.length(); // ← NullPointerException
// ✓ Validar antes: if (s != null)
```

CLASSCASTEXCEPTION

```
Object obj = "hola";
Integer i = (Integer) obj; // ← ClassCastException
// ✓ Validar: if (obj instanceof Integer)
```

TABLA RESUMEN

Excepción	Causa
ArithmeticException	División entera por 0
ArrayIndexOutOfBoundsException	Índice fuera de rango
InputMismatchException	Tipo de dato incorrecto en Scanner
NullPointerException	Método sobre objeto nulo

5 Buenas prácticas

✓ PATRONES CORRECTOS

✓ VALIDA ANTES DE OPERAR

if (divisor == 0) throw new ArithmeticException(...);
No esperes a que Java lo detecte automáticamente.

✓ USA VARIABLES LOCALES EN MÉTODOS

No sobrescribas los atributos de la clase con el input del usuario. Guarda el input en una variable local y compárala.

✓ CIERRA RECURSOS EN FINALLY

Scanner, ficheros y conexiones siempre en finally, nunca dentro del try.

✗ ERRORES FRECUENTES

Error	Solución
double / 0 sin catch	Validar con if antes
Código ejecutable fuera de método	Todo el código va dentro de un método
throw sin try-catch	Capturar siempre con catch
catch (Exception e) primero	Poner excepciones específicas antes
Comparar con == en String	Usar siempre .equals()

EXCEPCIÓN PERSONALIZADA

```
class SaldoInsuficienteException
    extends Exception {
    public SaldoInsuficienteException(String msg) {
        super(msg);
    }
}

// Uso:
throw new SaldoInsuficienteException(
    "Saldo: " + saldo);
```

★ REGLA DE ORO

Captura solo las excepciones que sepas gestionar. No uses catch (Exception e) {} vacío — oculta errores y dificulta el diagnóstico.